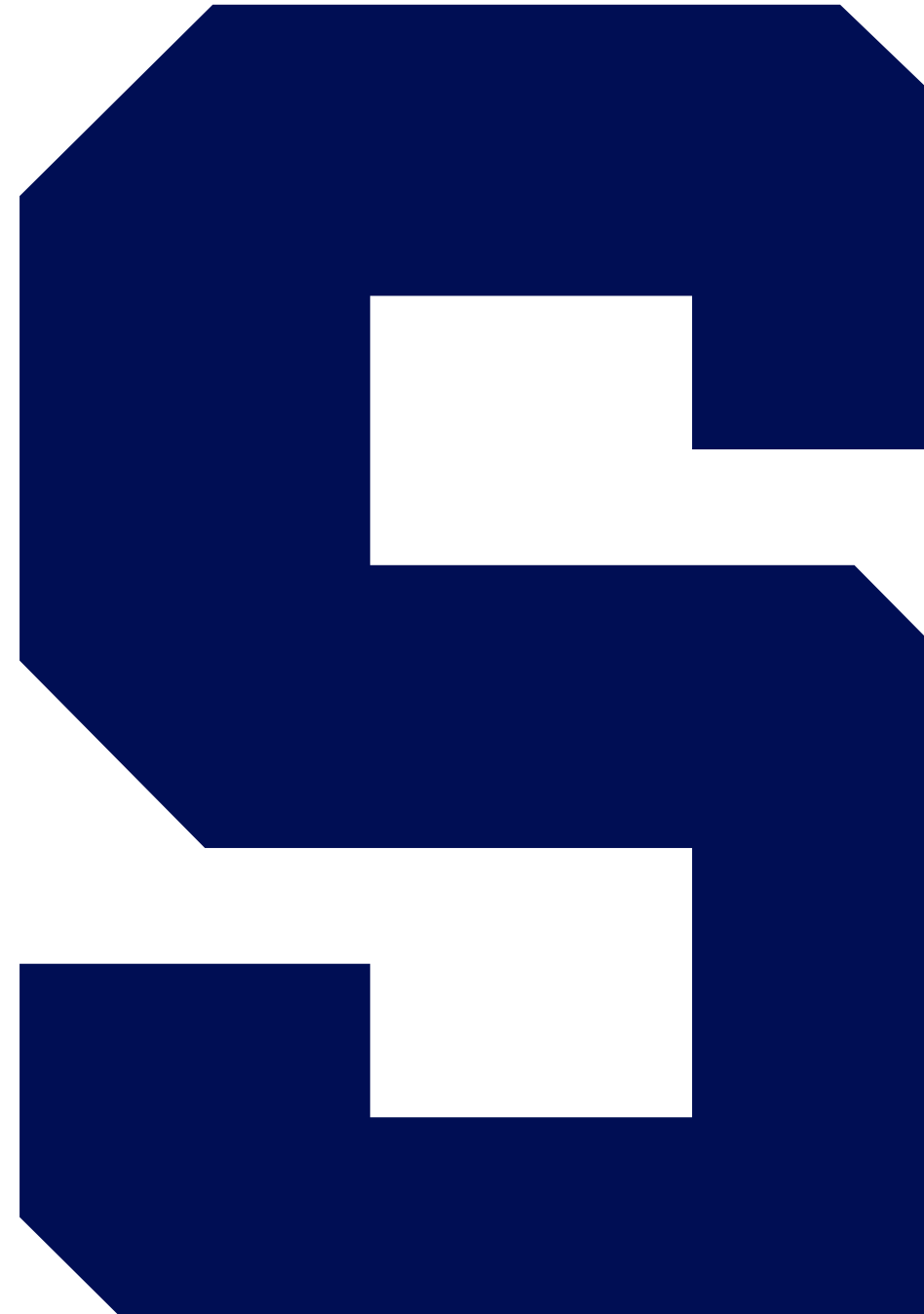
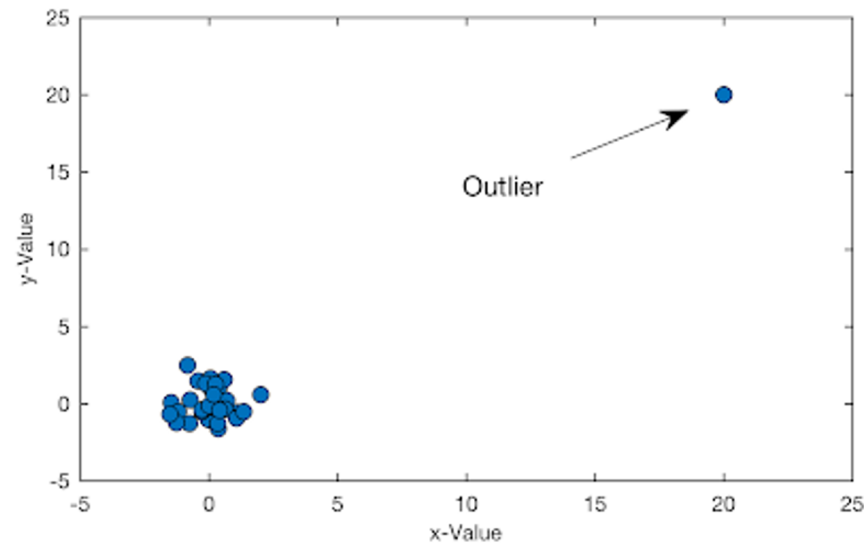


Data Preparation: Outliers



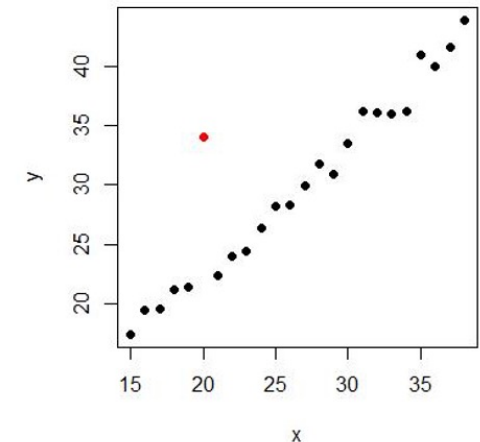
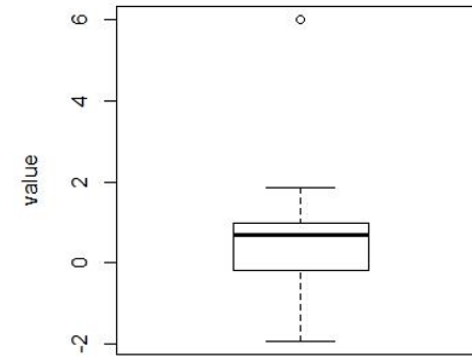
Outliers

- Outliers are data objects with characteristics that are considerably different than most of the other data objects in the data set



Detecting Outliers - Visual Inspection

- **Box Plots:** A box plot represents the distribution of the data and its central tendency. Points that are located outside the "whiskers" of the box plot are typically considered outliers.
- **Scatter Plots:** Useful to see the spread of data and identify potential outliers in bivariate analysis.
- **Histograms:** Helps to identify outliers in univariate analysis by visualizing the data distribution.



Detecting Outliers - Statistical Methods

- **Z-Score:** A Z-score represents the number of standard deviations a data point is from the mean. A high absolute value of Z-score (typically above 3) indicates that the data point is likely an outlier.
 - **IQR Method:** Outliers can be identified by finding values that lie below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$, where $Q1$ and $Q3$ are the first and third quartiles, respectively, and IQR is the interquartile range ($Q3 - Q1$).
-

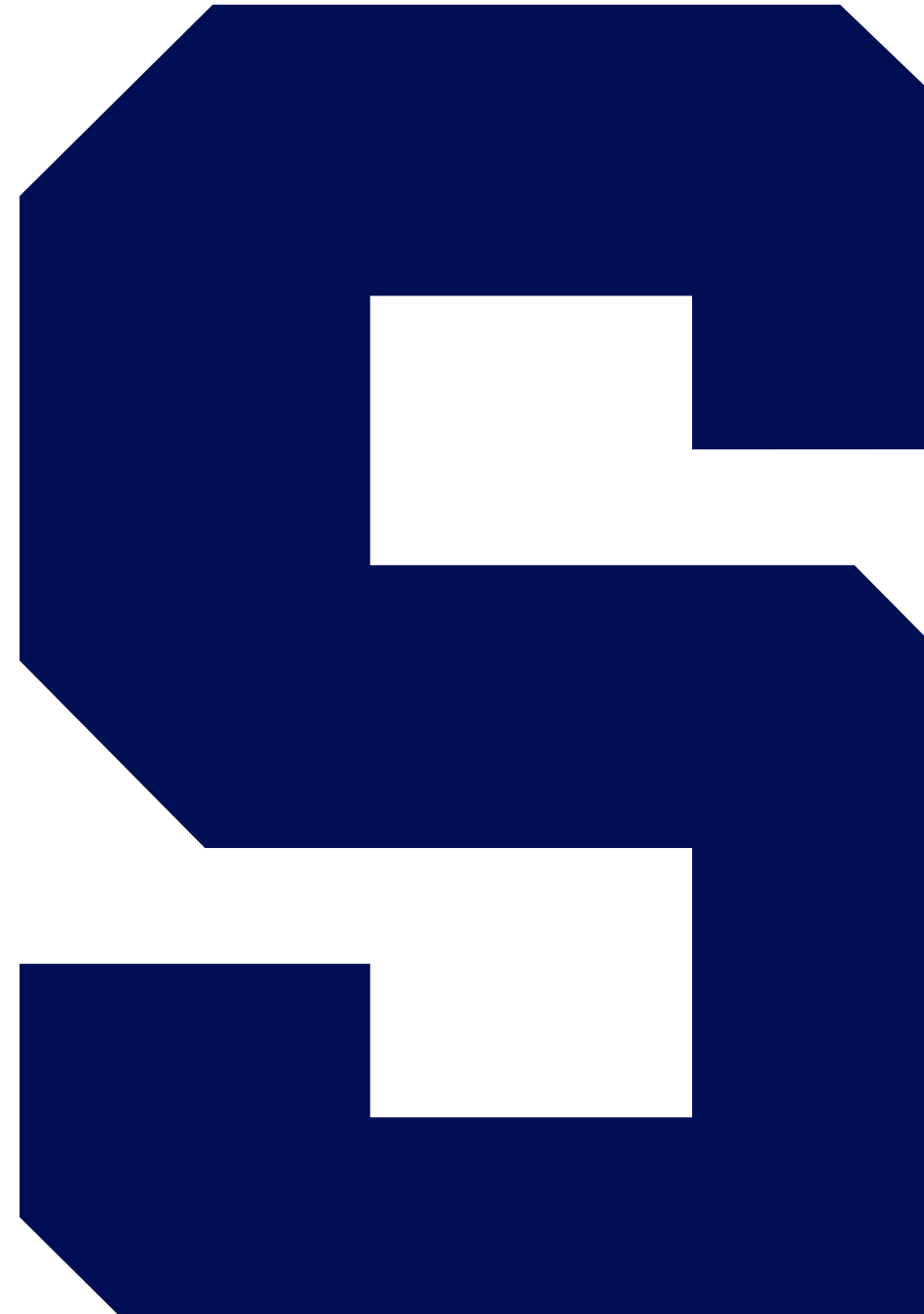
Handling Outliers

- Removal:
 - **Truncation:** Simply remove the outlier values.
 - **Winsorizing:** Cap the outlier values to a certain threshold.
 - Transformation:
 - **Log Transformation:** Reduces the impact of outliers but requires all values to be positive.
 - **Box-Cox Transformation:** Generalizes the power transformation and can stabilize variance and make the data more normal distribution-like.
-

Handling Outliers Cont.

- **Imputation:**
 - Replace outlier values with statistical measures such as mean, median, or mode.
 - **Robust Methods:**
 - Use models and methods that are robust to outliers, such as robust regression methods, tree-based models, or ensemble methods.
 - **Note:** Outliers aren't merely statistical anomalies — they are also values that are unrealistic. So, it's often a good idea to inspect your data carefully when you are trying to remove outliers.
-

Data Preparation: Scaling



Scaling

The price of many cryptocurrencies can be extremely volatile, rising and falling by hundreds or even thousands of a percent.

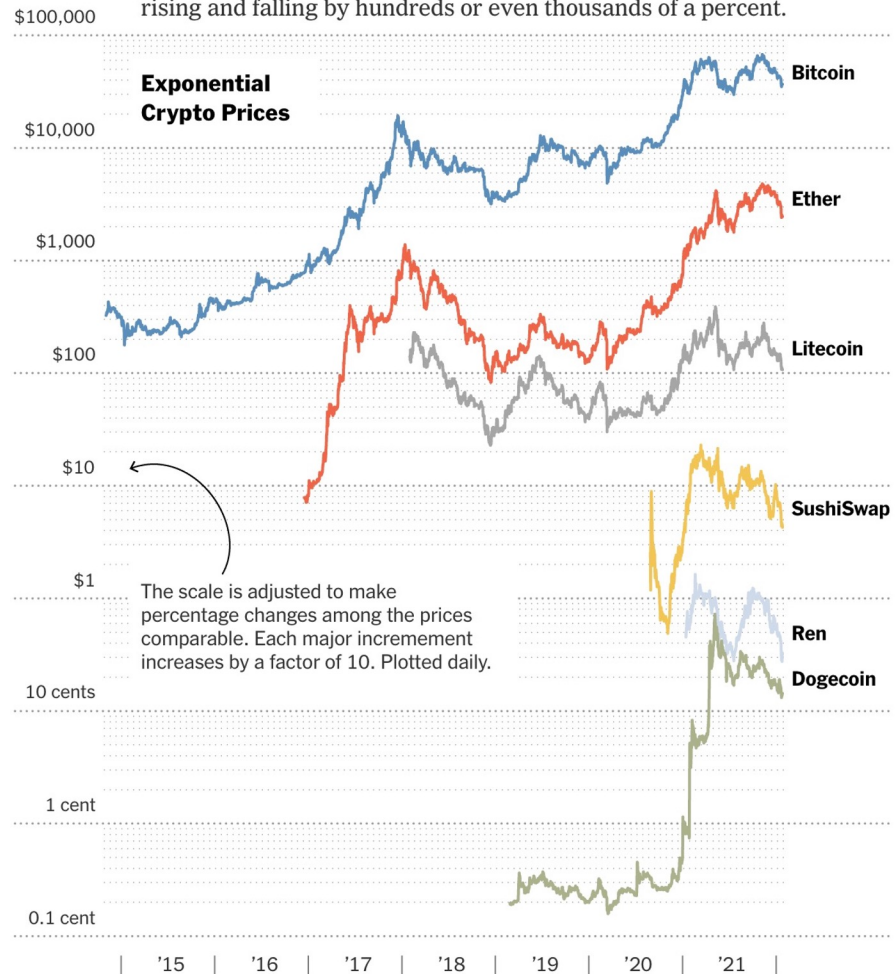
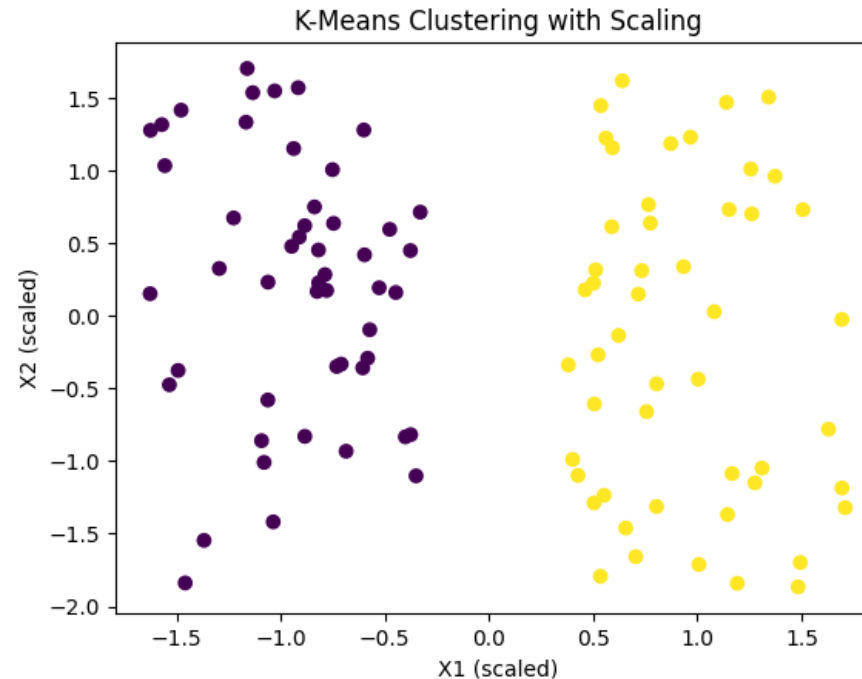
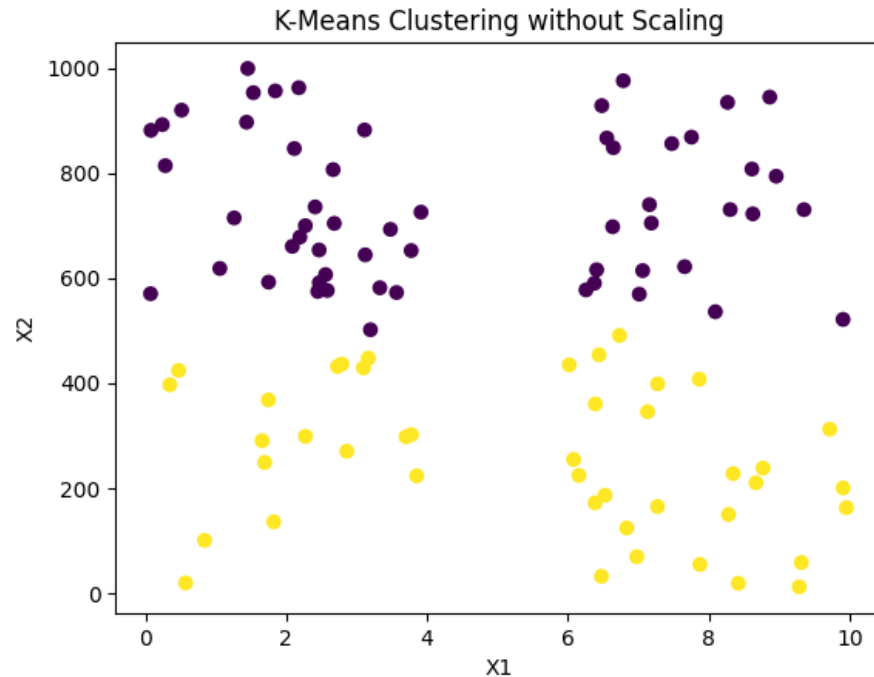


Image source: <https://www.nytimes.com/2022/01/27/business/crypto-price-bubble.html>

(Not) Scaling Can Affect Modeling

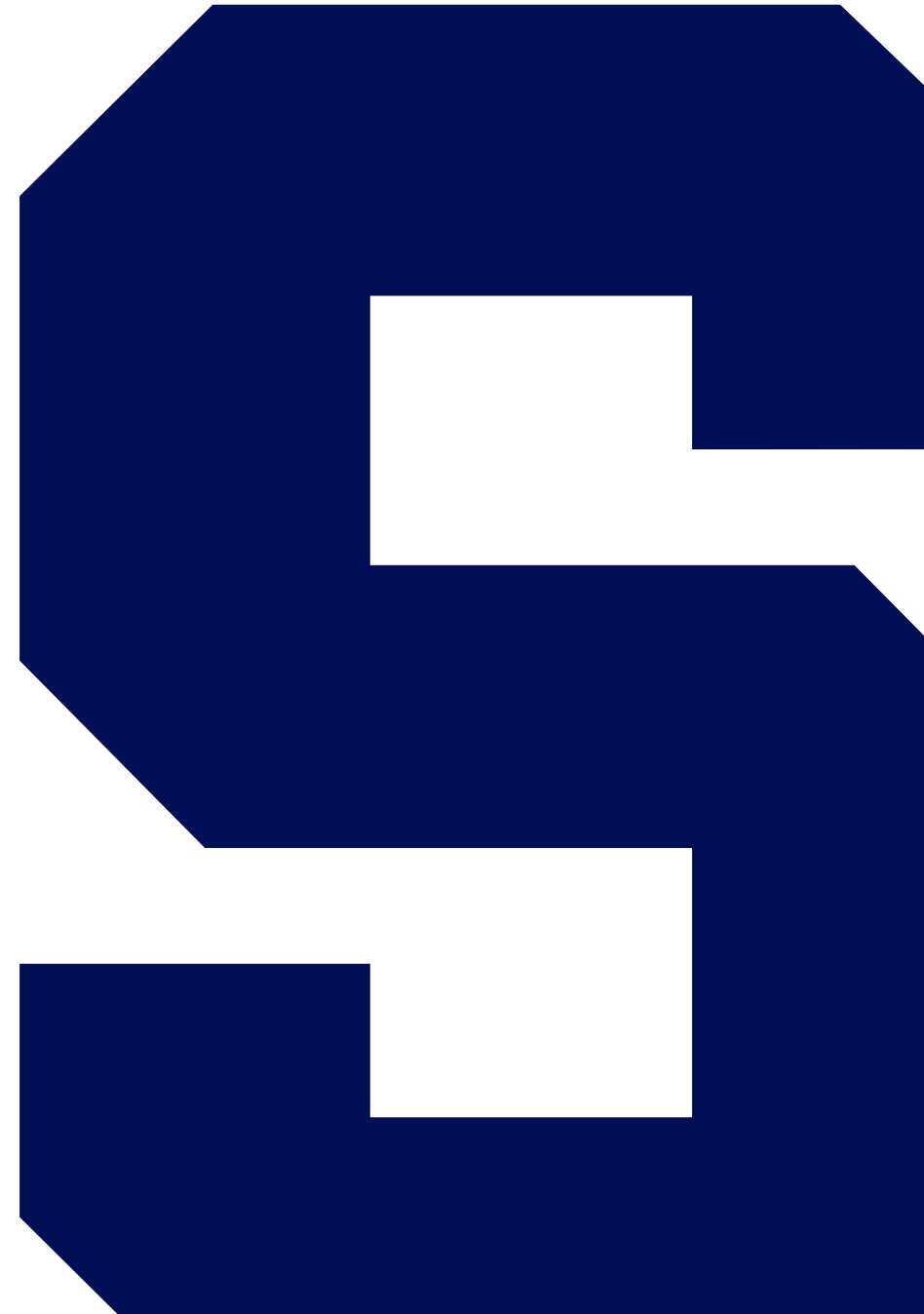
Many machine learning algorithms, such as K-Means clustering and support vector machines (SVM), use distance metrics like Euclidean distance to compare points in the feature space. Features with large numeric ranges can dominate the distance computation, thereby affecting the algorithm's performance.



How to Scale

- **Z-Scaling:** Use it when the feature roughly follows a normal distribution or when you don't have information about the distribution. It transforms the data into a distribution with a mean of 0 and a standard deviation of 1.
 - **Log-Scaling:** Useful for features that follow a power-law distribution. In these cases, log-scaling can help equalize the ranges and variances across features.
-

Data Preparation: Imputation

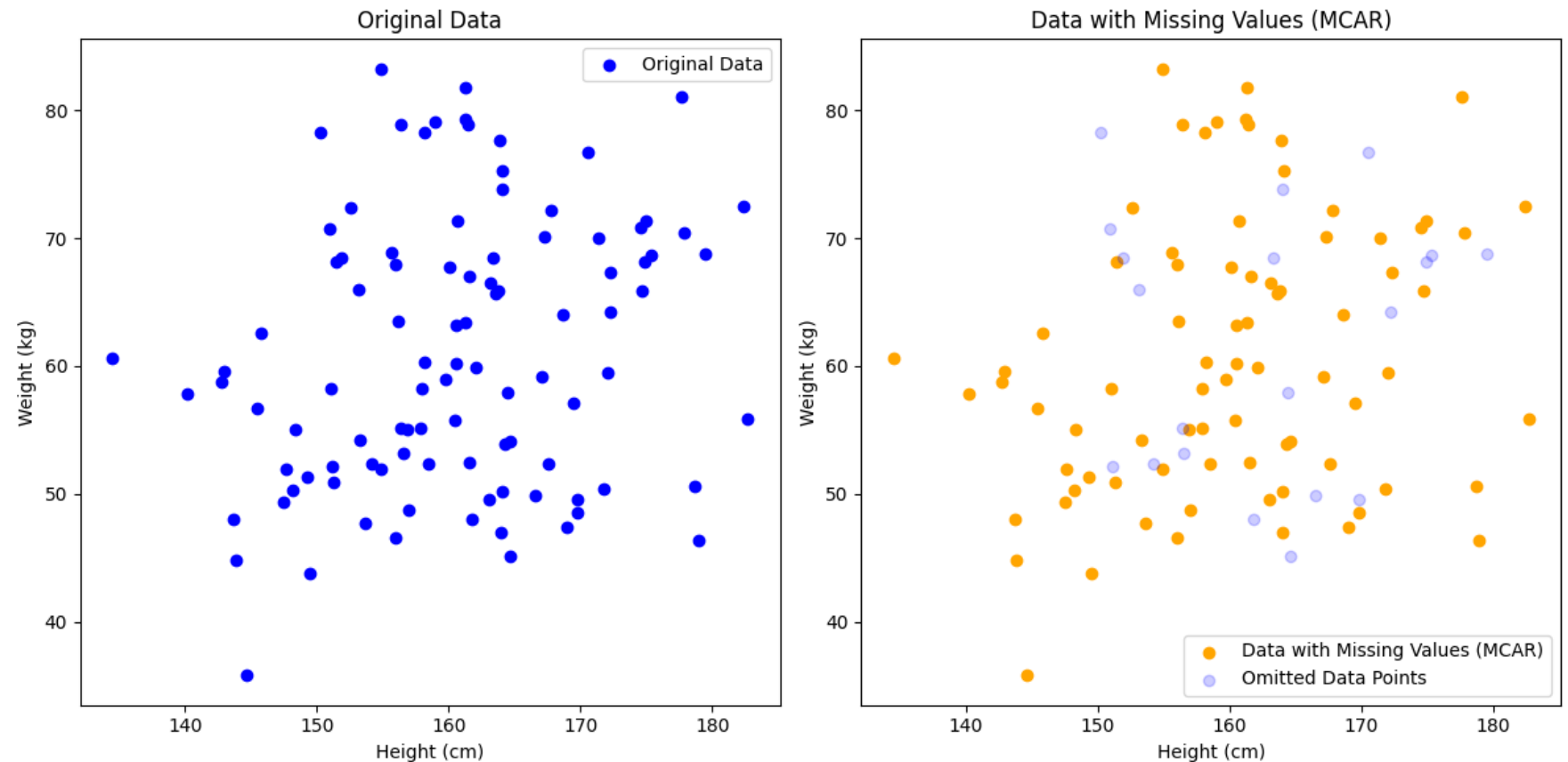


Imputation

- We use imputation methods to handle missing data.
 - There are three primary types of missing data:
 1. **Missing Completely at Random (MCAR)**
 2. **Missing at Random (MAR)**
 3. **Missing Not at Random (MNAR)**
 - We will briefly explore examples of each type.
 - We'll also discuss when to impute missing data.
 - Imputation methods are covered in demo.
-

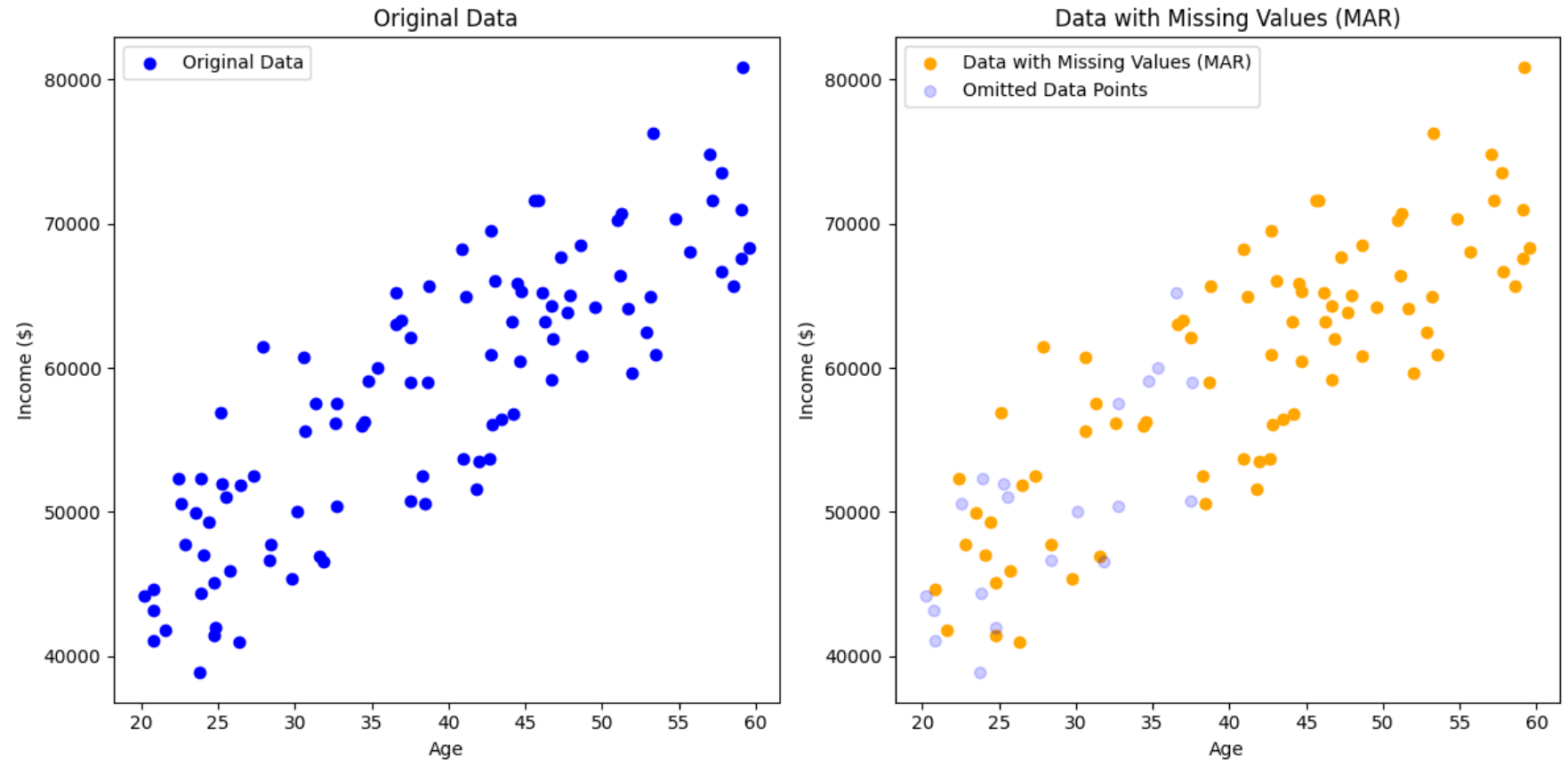
Missing Data: Missing Completely at Random

The missingness has **no relationship with any other variable**, observed or unobserved. It's random.



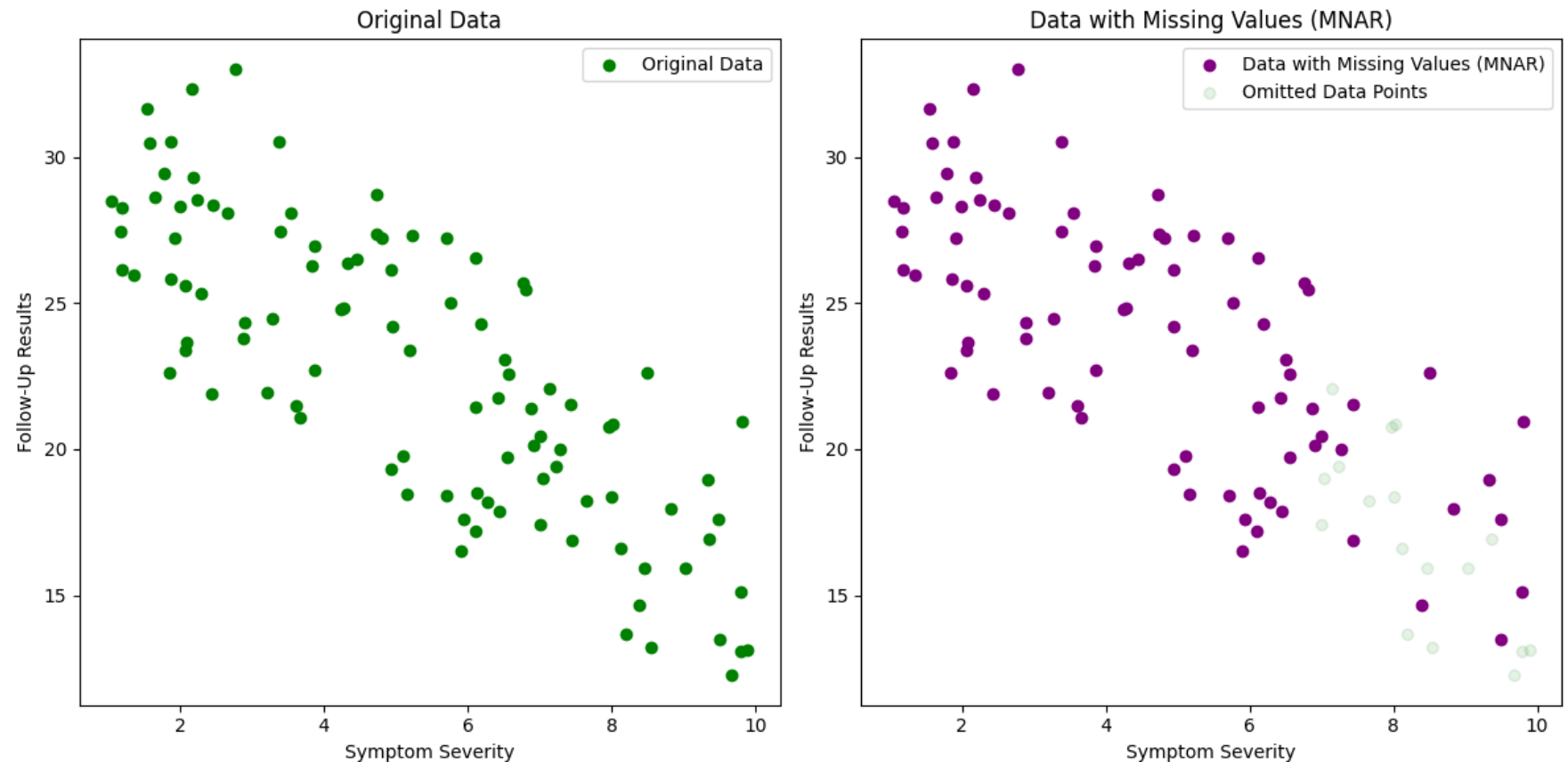
Missing Data: Missing at Random

The probability of **missingness** may be related to **observed data** but *not the missing data*.



Missing Data: Missing Not at Random

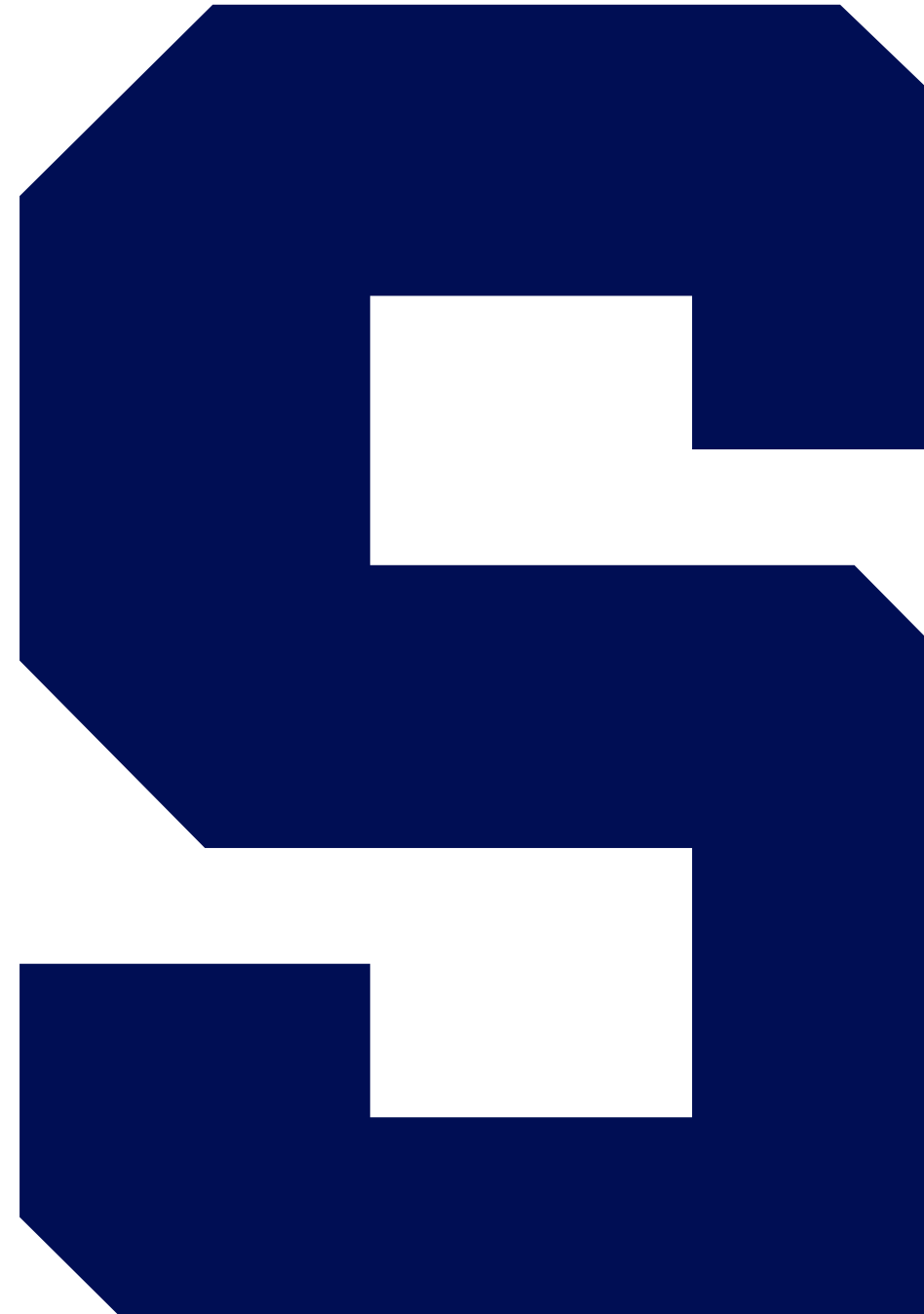
The probability of missingness is **related to the missing data itself** or a combination of missing and observed data.



Missing Data: When to Impute

- When to use imputed data:
 - When the amount of missing data is not substantial, imputation can leverage the available information, which would otherwise be discarded if only complete cases are used.
 - When the data are missing at random or missing completely at random, imputation can yield unbiased estimators.
 - When to avoid imputed data:
 - When missingness is related to the unobserved value itself (missing not at random), imputation might introduce bias.
 - When there are very few observed cases, imputation might overfit the training data, and it's better to use complete cases if available.
-

Data Preparation: Encoding



Encoding

- Many ML algorithms cannot use categorical data as is.
 - Encoding is the process of converting labeled (categorical) data to numbers.
 - Examples:
 - Nominal: Country, Vehicle type (SUV, sedan, motorcycle, truck)
 - Binary: Disease/No disease, Late departure/On-time departure
 - Ordinal: Clothing size ($S < M < L < XL$) , Income level (Low < Medium < High)
-

Encoding Examples

Ordinal

education_level	education_level_encoded
High school	1
Associate	2
Bachelor's	3
Master's	4
Doctorate	5

Binary

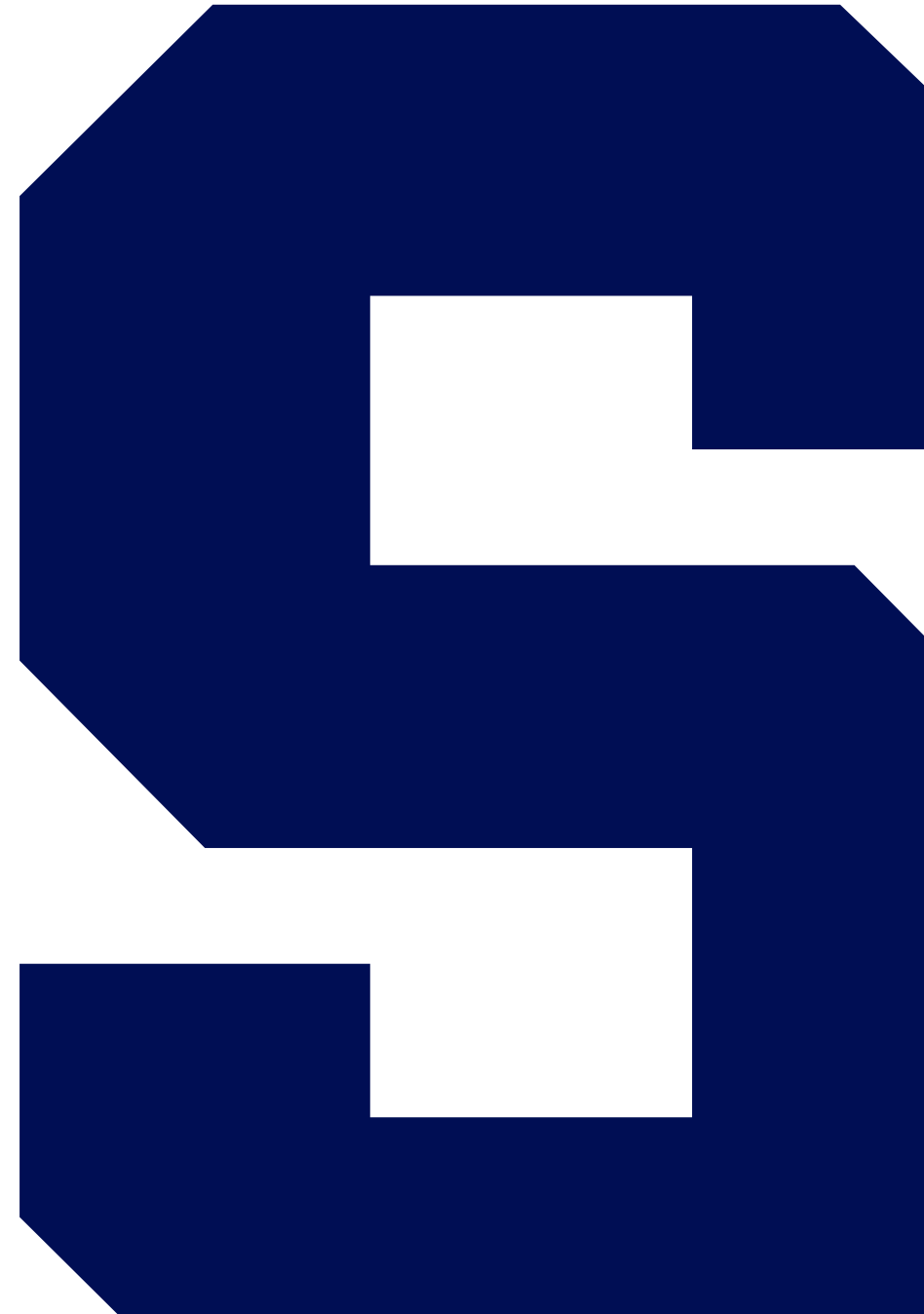
system_status	system_on
On	1
On	1
Off	0
On	1
Off	0

Encoding Examples Cont.

One Hot Encoding

blood_type	blood_type_A	blood_type_B	blood_type_AB	blood_type_O
A	1	0	0	0
B	0	1	0	0
AB	0	0	1	0
O	0	0	0	1

Data Preparation: Imbalanced Classes



Imbalanced Classes

- Occur when the number of samples in different classes of the target variable of a dataset are significantly disproportionate.
 - The imbalance can lead machine learning models to perform poorly on predicting the minority class.
 - Using metrics beyond simply accuracy (we will discuss later) can help, as can **resampling** techniques.
-

Resampling

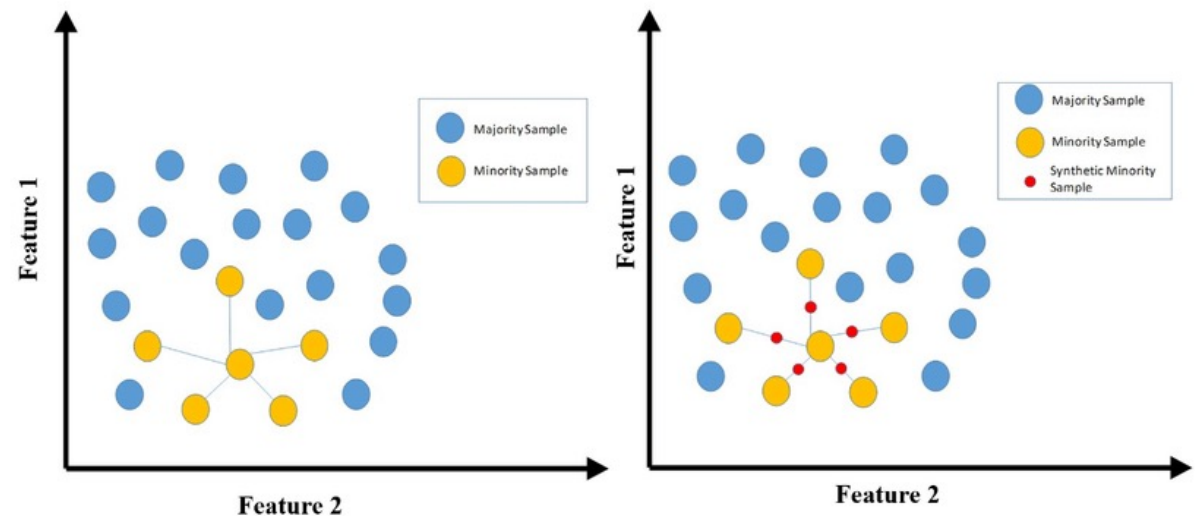
- **Random Over Sampling (ROS)**: Randomly **duplicate** samples in the minority class.
- **Random Under Sampling (RUS)**: Randomly **remove** examples in the majority class.
- **Synthetic Minority Over-sampling Technique (SMOTE)**: Generate new examples in the minority class by **interpolating** between existing examples.

Note: These methods can introduce bias or noise. As with all machine learning techniques, domain knowledge is crucial in choosing the right resampling method.

SMOTE

Approach:

- Randomly pick a point from the minority class.
- Calculate the k -nearest neighbors (for some pre-specified k) for this point.
- Choose one of the k -nearest neighbors and place a synthetic point anywhere on the line joining the point under consideration and its chosen neighbor.



SMOTE Benefits

- **Alleviates Overfitting:** Unlike random oversampling, SMOTE generates synthetic examples, reducing the risk of overfitting that comes with duplicating minority class examples.
 - **Class Separability:** It can enhance the decision boundary by filling in the gaps between minority examples, making it easier for classifiers to differentiate between classes.
 - **Improved Accuracy:** Often results in better classifier performance when compared to random under sampling or over sampling.
-

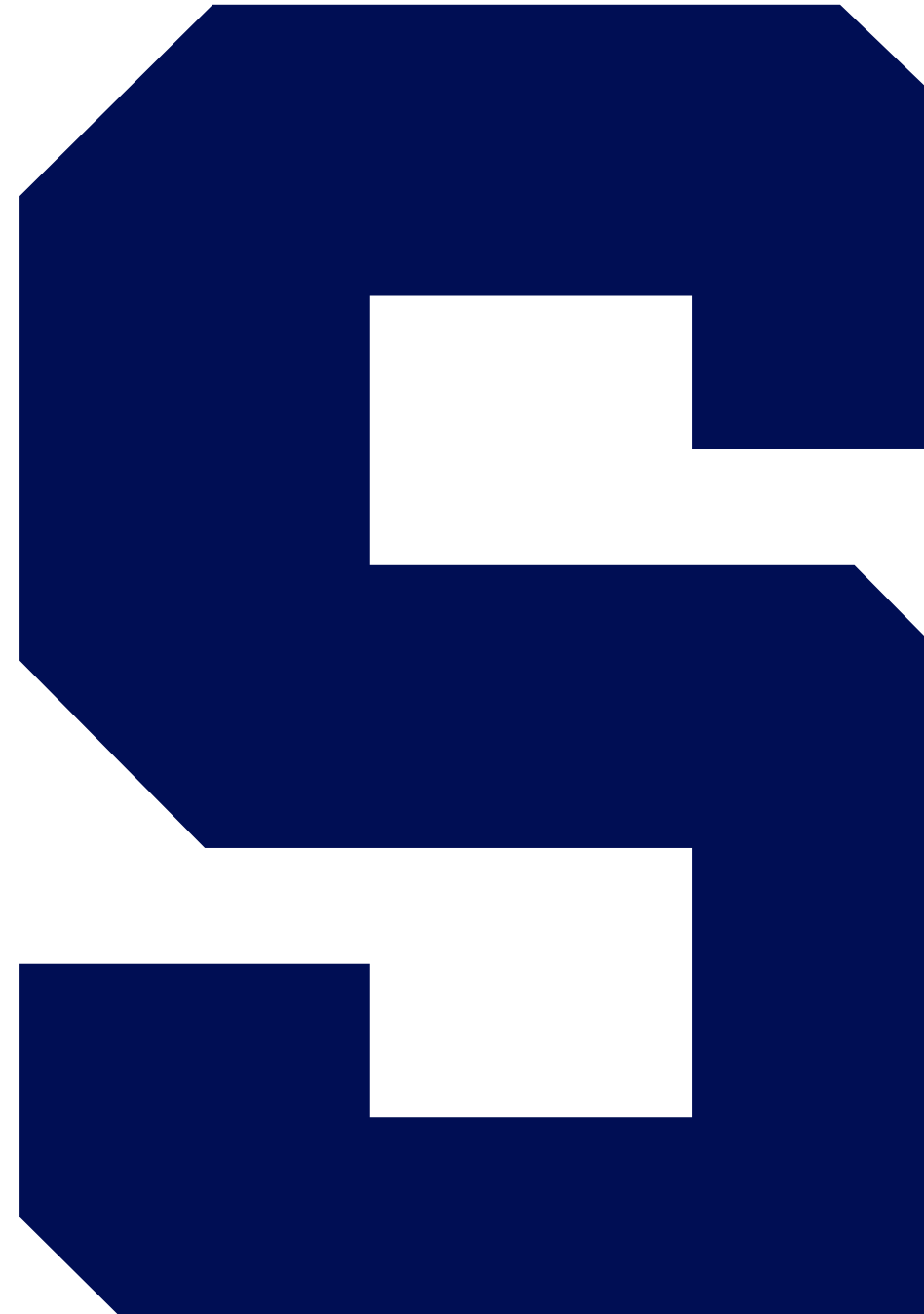
SMOTE Drawbacks

- **Introduces noise:** Synthetic examples created can be noisy and may not always correspond to valid examples, especially when the feature space is not continuous.
 - **Computational complexity:** Requires the calculation of k-nearest neighbors.
 - **Risk of over-generalization:** Can be too sensitive to the minority class, leading to higher false positives.
 - **Not ideal for high-dimensional data:** The concept of "neighbor" becomes less distinct in higher dimensions.
 - **May distort data distribution:** It can change the underlying distribution of the minority class and introduce a bias.
 - **Not suitable for all datasets:** Using SMOTE may not lead to a significant improvement in performance.
-

General Guidelines for Resampling

- Resampling by over-sampling can yield “brittle” classifiers that do not generalize the minority class.
 - Do not resample with test data included! This can lead to data leakage.
 - Whether or not to report results on resampled data depends on your goals:
 - **Yes:** when comparing different classifiers, or different resampling methods, or consider new algorithms.
 - **No:** when trying to show generalization, or in safety critical scenarios.
 - Oftentimes, a good strategy is to report both!
-

Model Evaluation Overview



Model Evaluation

A critical step in a machine learning workflow is evaluating your model.

There are two important considerations when evaluating a classifier:

1. How to sample your data for testing.
 2. What measurements to use.
-

Training Vs. Generalization Error

- **Training error:** The error that the model makes on the training data, the same data it learns from.
 - Learning the training data too well can lead to overfitting.
 - **Generalization error:** Measures how well a model performs on unseen data, that is, data not used during the training process.
 - The goal in model development is to minimize both training error and generalization error to create a model that is accurate and robust against unseen data.
-

Accuracy is Not Always Enough

- **Accuracy:** The number of correct predictions (both true positives and true negatives) divided by the total number of predictions made.
- Accuracy can be misleading, especially in cases where the dataset is imbalanced.

		PREDICTED CLASS	
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	A (TP)	B (FN)
	Class=No	C (FP)	D (TN)

$$\text{Accuracy} = \frac{A + D}{A + B + C + D} = \frac{TP + TN}{TP + TN + FP + FN}$$

Sampling Data

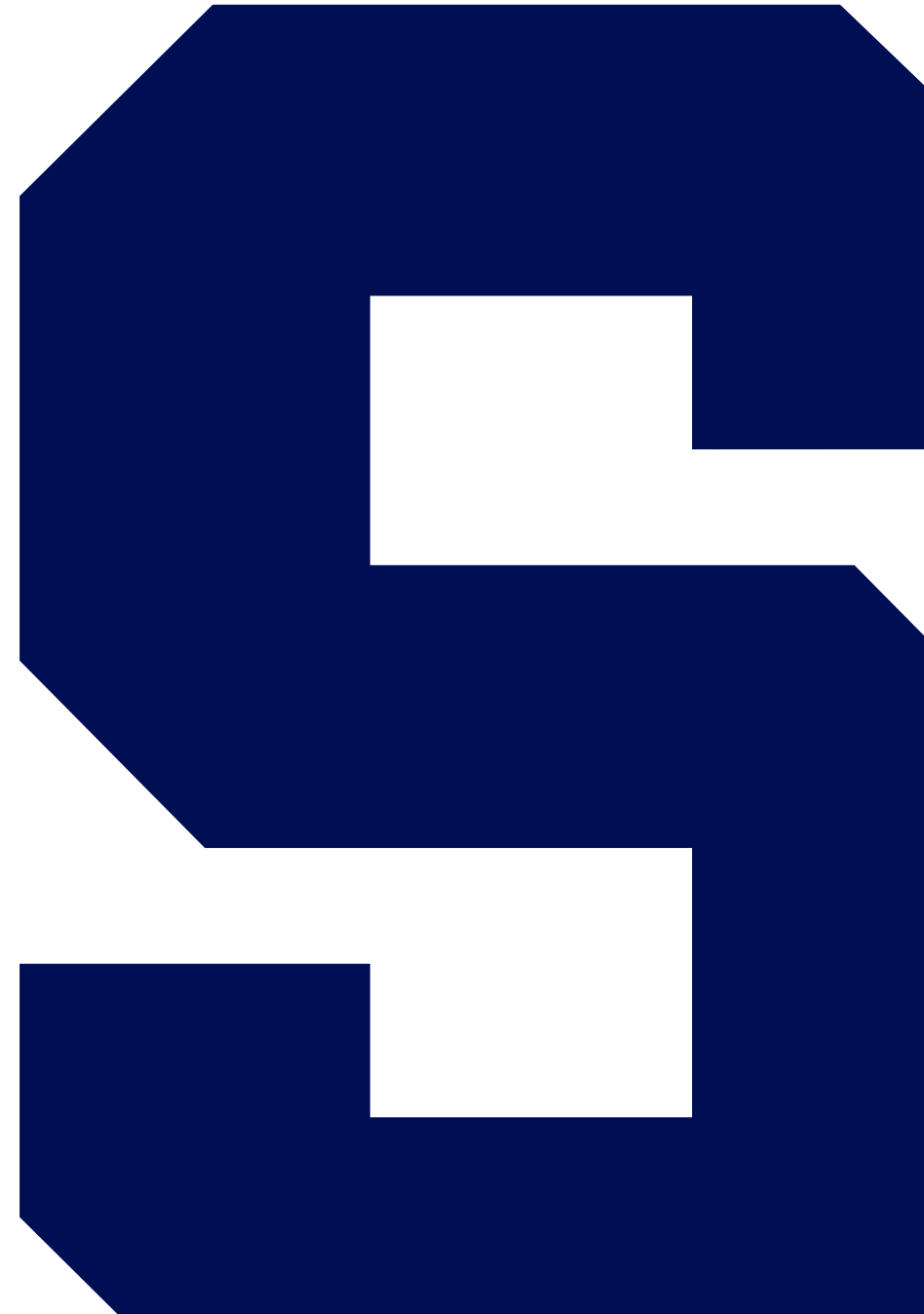
Train-Test Split

- **Basic idea:** Separate your data into two sets - one for “training” the model, and the other for “testing” the model.
 - **Important considerations:**
 - The training set must be completely **independent** and highly **representative** of the test set to get an unbiased estimate of performance.
 - Performance can vary depending on the split, and a bad split could result in poorer performance.
 - Example: Using data from one geographic region in the training data and another region in the test set.
-

Stratification

- **Stratification:** A technique used in data splitting to ensure each subset of the data has a similar distribution of class labels.
 - Stratification is particularly crucial when dealing with imbalanced datasets, where one or more classes are underrepresented compared to others.
 - Splitting data without stratification can lead to biased or inaccurate models, as the model may not be trained or evaluated on a representative sample of data.
-

Cross-Validation



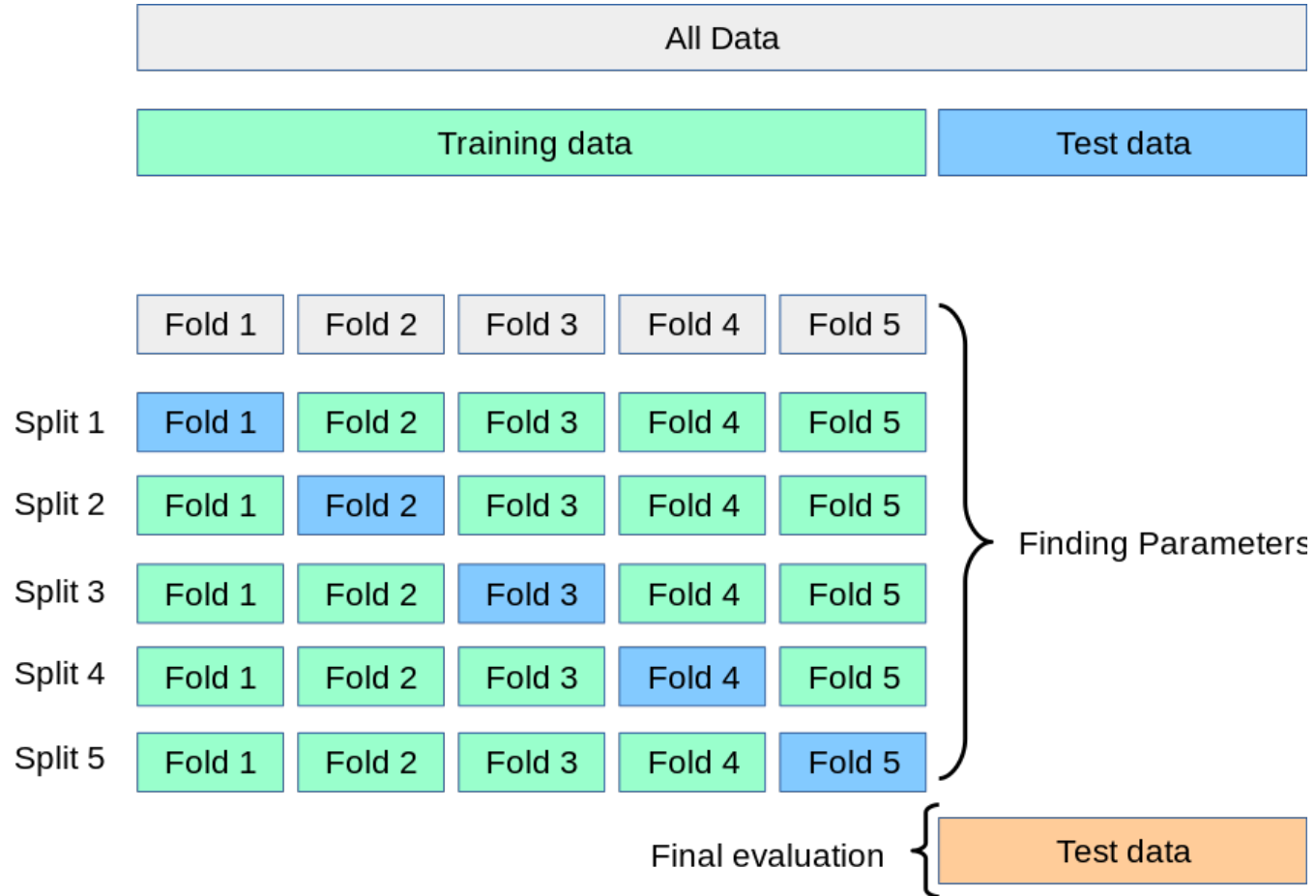
Cross-Validation

- **Cross-validation:** A technique where a dataset is divided into multiple subsets or "folds." The model is trained and evaluated repeatedly, with each fold serving as the test set in turn, while the others are used for training.
 - The final measure of model performance is the average across all folds, enhancing reliability and reducing dependence on any single data split.
 - Cross-validation helps in mitigating variability inherent in relying on a single train-test split.
 - Every data point is used for both training and testing, making evaluation more reliable.
-

Cross-Validation Methods

- **K-fold cross-validation:**
 - Partition data into k subsets or "folds."
 - Train on $k-1$ of these folds and test on the remaining fold.
 - Repeat this process k times, average performance metrics.
 - **Leave-one-out cross-validation:**
 - Extreme cross-validation - train on all available data, holding back just one case for testing.
 - Computationally very expensive.
 - **Stratified k-fold cross-validation:**
 - Each fold is stratified.
-

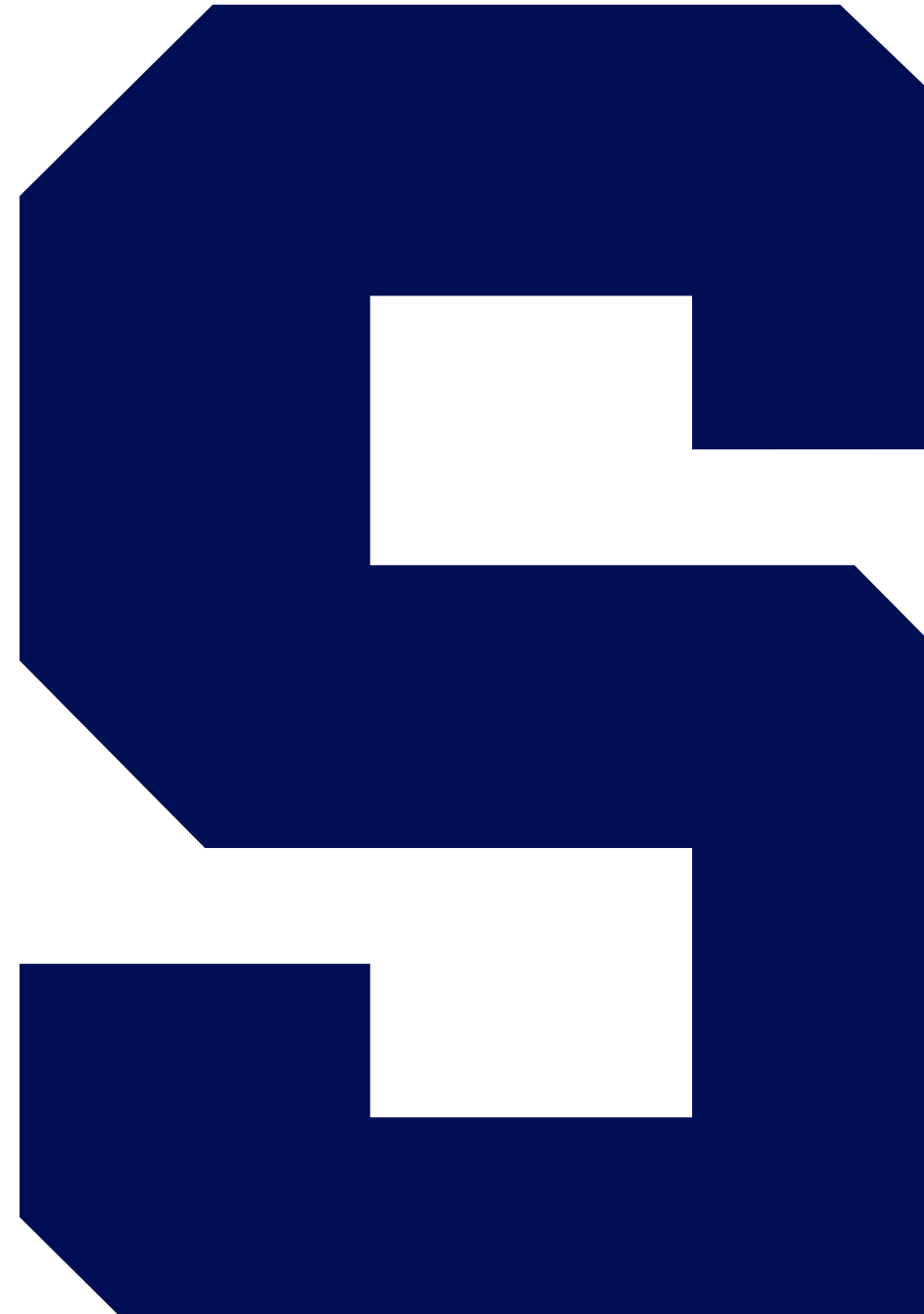
Cross-Validation Illustration



Cross-Validation Considerations

- Not appropriate for time-series data (use time-series specific cross-validation).
 - A greater number of folds increases available training data but...
 - Increases processing time and performance variance.
 - Reduces representativeness of test samples.
 - Data leakage if any preprocessing/feature selection is done after splitting but before training. All such operations need to take place on the training set.
-

Metrics for Evaluation



Confusion Matrices

- **True Positives (TP):** These are the correctly predicted positive observations.
- **True Negatives (TN):** These are the correctly predicted negative observations.
- **False Positives (FP):** Incorrectly predicted positive observations (Type I error).
- **False Negatives (FN):** Incorrectly predicted negative observations (Type II error).

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
Class=Yes	A (TP)	B (FN)
	C (FP)	D (TN)

Precision

- **What It Measures:** Precision quantifies the accuracy of the model in predicting positive instances. It's the ratio of true positives (correctly predicted positives) to the total number of predicted positives (both true positives and false positives).
- **Interpretation:** A high precision indicates that the model is reliable in its positive predictions, but it doesn't tell us how many actual positives were missed.

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	A (TP)	B (FN)
	Class=No	C (FP)	D (TN)

$$\text{Precision} = \frac{A}{A + C} = \frac{TP}{TP + FP}$$

Recall (Sensitivity)

- **What It Measures:** Recall measures the model's ability to detect positive instances among all actual positives. It's the ratio of true positives to the actual positives in the data (both true positives and false negatives).
- **Interpretation:** High recall means that the model is good at detecting positive instances, but it may also be including some false positives.

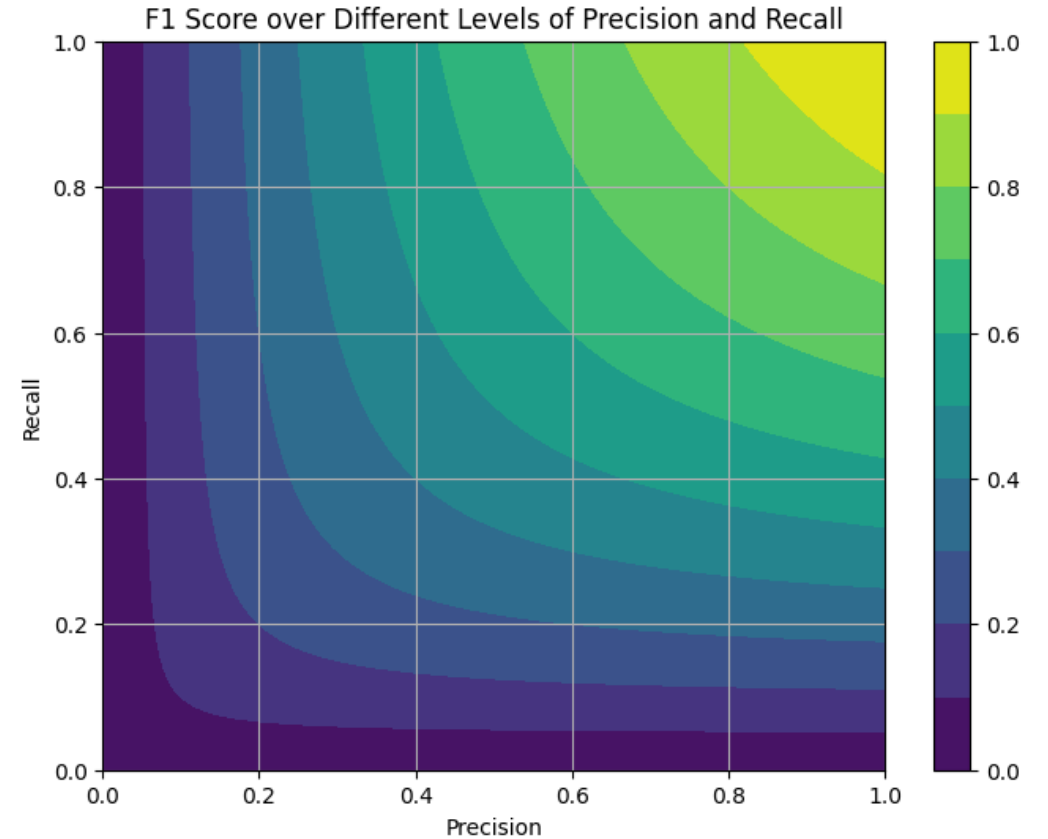
ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
	Class=Yes	Class=No
	A (TP)	B (FN)
	C (FP)	D (TN)

$$\text{Recall} = \frac{A}{A + B} = \frac{TP}{TP + FN}$$

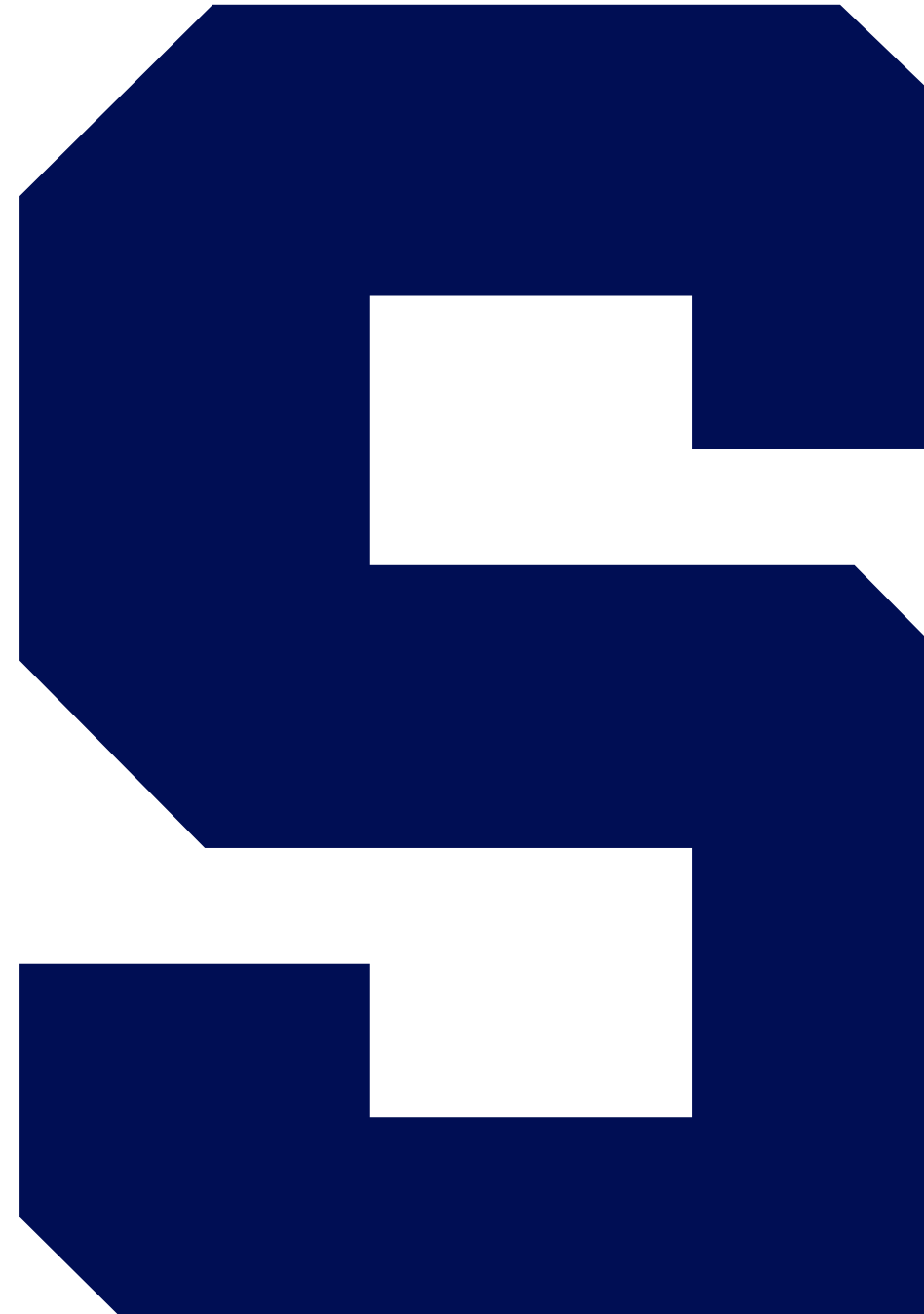
F-Score

- **What It Measures:** The F-Score is the harmonic mean of precision and recall. It provides a single score that balances both the concerns of precision and recall in one number.
- **Interpretation:** A high F-Score suggests a balanced trade-off between precision and recall. It is particularly useful when you seek a balance between these two metrics and when the class distribution is imbalanced.

$$F_1 = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$



ROC Curves



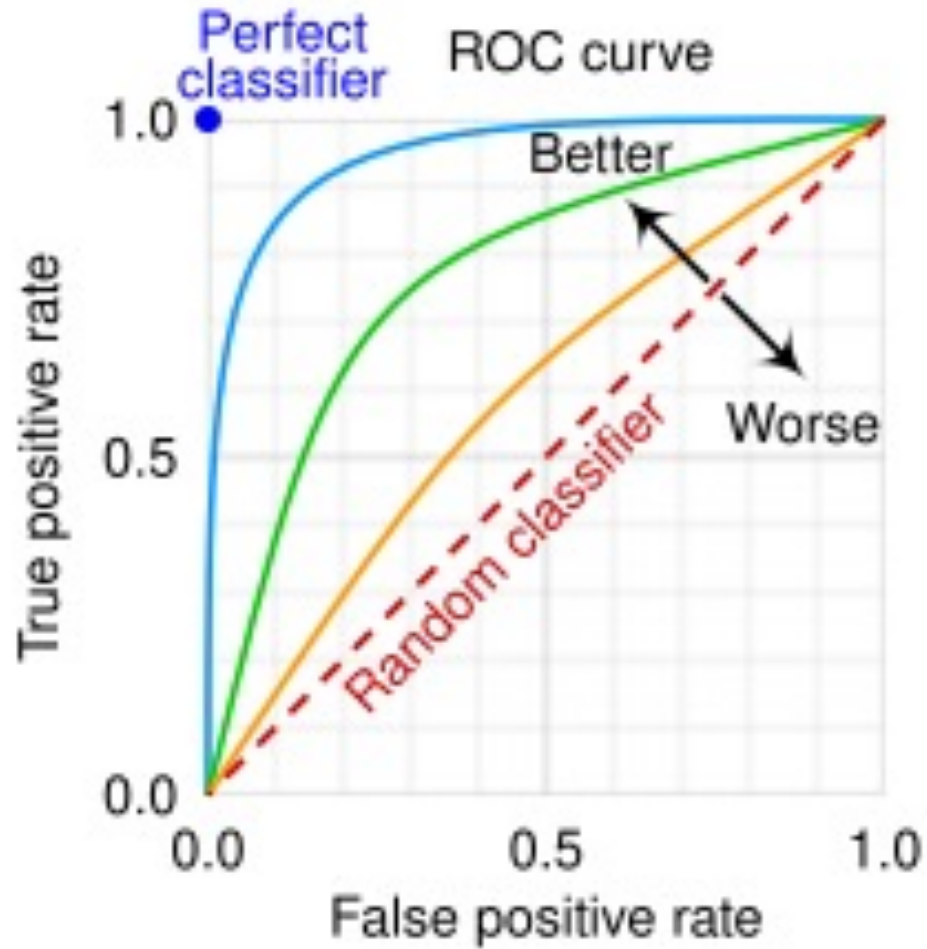
ROC Curves

A **Receiver Operator Characteristic** (ROC) curve is a graphical representation of a classifier's performance across all possible **thresholds**. It plots two parameters:

- **True Positive Rate** (TPR), also known as **Recall**, on the y-axis.
- **False Positive Rate** (FPR), also known as **Fallout**, on the x-axis.

Interpreting the Curve: The ROC curve shows the tradeoff between sensitivity (or TPR) and specificity ($1 - \text{FPR}$). A higher curve indicates a better performance. An ideal classifier would have a curve that goes straight up the y-axis and then along the x-axis.

ROC Curve Illustration



Area Under the Curve (AUC)

AUC provides a single numeric metric to summarize the ROC curve. It represents the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative one.

Interpretation:

- An AUC of **1** indicates a **perfect classifier**.
- An AUC of **0.5** suggests a **no-skill classifier**, equivalent to random guessing.
- AUC values between 0.5 and 1 indicate different levels of classifier performance, with higher values signifying better classification.

Both ROC curves and AUC are particularly useful for evaluating classification models in the presence of class imbalance or when dealing with probabilistic predictions.
